Aran - Ledger, AI Gateway and Red Team

Aran: mobile and OTP fixes, leakage ledger, AI gateway, red team and speed

Roadmap items: 3 (mobile and OTP weak spots), 6 (AI gateway), 9 (red team), M4 (leakage ledger, from the research doc), and a first step on 7 and 12 (NER cascade and speed). Items 1 and 4 (live console, crypto-shredding) were built in parallel and are on main as `app/` and `research/firewall/shredder.py`.

Owners: Bharadhwaj M and SN Arun Vijay (item 3, AI gateway, NER speed); Inbathamizhan S and Aasif Ahmed A (ledger, red team, gateway access control and audit log).

Read first: Learn 12: Leakage Ledger, AI Gateway and Red-Teaming.

Data. Real logs are the 16 Loghub systems (models train on 8, every test uses the other 8) and the UCI SMS corpus; NER corpora are CoNLL-2003, HiNER and Hinglish tweets. Every Aadhaar, card, mobile, PAN, OTP and UPI ID is generated (valid checksums, belongs to no one). Raw numbers are in `research/results_ai.json`, `results_registry.json` and `results_next.json`; the code is `research/experiment_next.py`.

Summary

What	Before	After
Sensitive values removed by the firewall, no registries (E9)	83.5%	**91.0%**
Mobiles with no cue word, no registry (E9)	69.8%	**100%**
OTPs next to transaction words, no registry (E9)	79.4%	**100%**
Mobiles / OTPs on phrasings written after the fix (E13b)	67.2% / 94.3%	**100%** / 84.7%
Reference numbers wrongly taken as mobiles on those phrasings (E13b)	0%	17.3%
Clean real log lines wrongly changed (E9, 14,890 lines)	122	150 (130 with the NER gate)
Customers left below 20 bits in some vendor's joined view (E13)	14,175 of 60,000 identifiers	**0**
Raw values reaching an LLM provider (E14, 2,000 prompts)	100%	**7.8%** (names and addresses only; every number 0%)
Prompt-injection and hijack attacks that leaked a value (E14)		**0** of 5 attack types
Tricks fully caught out of 26 (E15)	14 (E10 normaliser)	**22**

What	Before	After
Detector crashes on mixed-script numbers (E15)	125 of 500 lines	**0**
Lines per second, same output (E16)	274	348 cached, **654** with the gate

Two results went the wrong way and are reported as they are: OTP recall on the new phrasings fell from 94.3% to 84.7%, and 17% of reference numbers after "cheque no" were taken as mobiles. Section 1 explains both.

1. Mobile and OTP weak spots (item 3)

What changed. Three mobile features in `ai/detector.py` (a phone-named JSON or `key=value` field, a word that usually precedes a phone number, a word that announces a reference number), four OTP context features in `ai/context_clf.py` (the nearest cue on the left, an OTP word in the same clause, both cues together, a connector such as "is" or ":"), and `synth.hard_line`, which adds 8% training lines with OTPs next to transaction words, the same shapes without an OTP word, bare mobiles after a verb or in a phone field, and 10-digit reference numbers. All of it uses TRAIN phrasings only.

E9 (8 unseen real systems, 16,000 lines, 1,895 injected values, no registries):

Type	Left before	Left after
Mobile	57/189	**0/189**
OTP	60/291	**0/291**
Name	170/643	146/643
Address	26/254	24/254
Aadhaar, card, PAN, UPI	0	0
All	83.5% removed	**91.0%** removed

E6 OTP recall on English test lines rose from 64.8% to 100% at precision 1.0, and the calibrated mobile detector's AUROC from 0.946 to 0.981 (E7). With these fixes, a bank no longer needs the own-customer and OTP registries to close these two gaps: E11 gives 91.0% with or without them. The registries still matter for typo repair (E12, 99.2%) and as stronger evidence when phrasing is unusual.

Cost. Clean real lines wrongly changed rose from 122 to 150 of 14,890 (0.82% to 1.01%). The extra changes are UPI IDs (101 to 117, user@host strings in Thunderbird logs) and person names (12 to 31, from the retrained NER); OTP false alarms fell from 7 to 0. Total false findings on real logs rose from 60 to 95. The NER gate in section 5 brings changed lines back down to 130.

E13b: does it generalise? One TRAIN phrasing ("{OTP word} for {ref word} {id} is {value}") has the same shape as the TEST template that used to fail, so E9 alone could flatter the fix. Twelve new phrasings were written after the fix, appearing in neither TRAIN nor TEST, with 600 positives and 600 negatives per type:

	Old models	New models
OTP recall	94.3%	84.7%
OTP false alarms	0%	0%

	Old models	New models
Mobile recall	67.2%	**100%**
Mobile false alarms	0%	17.3%

Both regressions come from one phrasing each. "{OTP} is the PIN to authorise txn {id}" (value first, transaction word after) is missed every time, and "cheque no {number}" is always taken as a mobile, because "no" was put in the list of words that come before a phone number when it also announces reference numbers. These are the next fixes, and they need a third set of fresh phrasings to measure honestly, since this set is no longer unseen.

2. Leakage ledger (M4)

`research/firewall/ledger.py`. For every identifier and every trust domain (systems that can join their data), the ledger keeps the digit positions shown so far, counts the identifiers still consistent with them exactly (Verhoeff and Luhn dynamic programmes, closed form for mobiles) and refuses any disclosure that would leave a domain below τ bits. A refused mask falls back to a smaller mask inside the one requested, or a token. Entries are keyed by HMAC, so the ledger holds no identifiers. `AranFirewall(ledger=..., mask_policy=...)` uses it for sinks that ask for partial masks.

E13. 20,000 generated customers, 11 sinks in 4 trust domains, masks those systems typically ask for (last 4 for statements and CRM, first 6 or first 8 for BIN analytics, first 6 and last 4 for disputes), 167,654 disclosures in random order. τ = 20 bits for Aadhaar and cards, 16 for mobiles; the SMS gateway is exempt for mobiles.

	Identifiers below τ in some domain	Of those, 1,000 or fewer candidates	Disclosures shrunk
Masks as configured	14,175 (Aadhaar 1,934, card 10,423, mobile 1,818)	7,357	0
Per-field check (each mask alone ≥ τ)	13,053	7,357	1.2%
Aran ledger	**0**	**0**	9.7%

A per-field rule barely helps, because each mask is safe alone; the leak is in the join. The canonical case: vendor A's analytics system gets the first 8 digits of a card (23.3 bits left), its collections system asks for the last 4, and together they would leave 1,000 candidates (10.0 bits). The ledger gives collections a token. The most common shrinks were statement printing (last 4 to last 2, 3,473 times) and rewards (first 6 to first 4, 3,463), both inside vendor B. The cost is speed (61 µs per disclosure) and memory (8 MB for 264,042 entries). Min-entropy assumes identifiers are uniform; real BINs are not, so τ should keep a margin.

3. AI gateway (item 6)

`research/gateway/ai_gateway.py`, with `make_router(gateway)` for `POST /v1/chat/completions` (OpenAI-compatible) and `GET /v1/aran/audit`. Values become typed placeholders such as `<AADHAAR_1>` before the provider sees them; replies are scanned again; placeholders resolve only for the conversation's own caller and only for types the caller's role may see; every resolution goes into a hash-chained audit log.

E14. 2,000 staff prompts (six wordings such as "Summarise this customer complaint" around a TEST line), sent to a recording provider that echoes the prompt back, which is the hardest case because every placeholder returns.

Type	Raw values reaching the provider	Customer-care role sees in clear
Aadhaar	0/79	0/79 (last 4 only)
Card	0/68	0/68 (last 4 only)
PAN	0/79	0/79 (masked)
Mobile	0/225	225/225
UPI	0/439	439/439
OTP	0/406	0/406 (destroyed, never stored)
Name	161/793	793/793
Address	26/304	298/304

Without the gateway all 2,393 values reach the provider; with it 187 (7.8%) do, all of them names and addresses the NER missed. Every number type is at 0. The analyst role saw only those same 187 missed values in clear. Real SMS wrongly changed: 16 of 2,000 (15 names, 2 addresses). Tokenising costs 2.5 ms per prompt (p95 3.7 ms).

For the KYC officer, who may see every type, 1,900 of 2,000 echoed replies came back exactly as typed (OTPs aside, which are destroyed on purpose). In the other 100 every value came back, but an Aadhaar or card number typed with spaces or dashes came back as plain digits, because the vault stores the canonical digits so that one number written two ways gets one placeholder.

Attack	Result
A1 another caller asks the model for the victim's placeholders	0/79 values leaked
A2 a placeholder is planted in the prompt before a real value appears	0/79 Aadhaar shown in clear
A3 reuse of another caller's conversation id	79/79 refused
A4 the model produces real-looking Aadhaar and card numbers itself	0/400 shown to the caller
A5 Aadhaar hidden in the prompt with Devanagari, dots, \u escapes or base64	0/316 reached the provider
Editing one audit entry	detected (chain no longer verifies)

The provider here is a recorder, not a real model; the gateway code is the same for any OpenAI-compatible provider through `openai_upstream`. A real model may rephrase placeholders ("AADHAAR 1"), which the recorder never does; that needs a test with a live model.

4. Red team (item 9)

E15 hides Aadhaar, card and mobile numbers in 500 TEST lines 26 ways and counts a value as removed only if it cannot be recovered from the output by undoing escapes twice, other scripts, number words, look-alike letters, letters between digits, separators, base64 twice and hexadecimal, read forwards or backwards. 625 values per trick.

Trick	Detector only	E10 normaliser	+ registry	+ red-team additions
12 E10 tricks (scripts, escapes, separators, zero-width, split fields, base64)	1 of 12	12 of 12	12 of 12	12 of 12
Followed by a full stop	100% (0% before the fix)	100%	100%	100%
Mixed scripts in one number	0% (crashed 125 of 500 lines before the fix)	100%	100%	100%
Number words: English, with double/triple, Hindi, Tamil, Hinglish	0%	0%	0%	**100%**
Look-alike letters (O, l, S, B)	0.8%	0.8%	0.8%	**100%**
Double URL-encoding	0%	0%	0%	**100%**
One-digit typo	60%	60%	**99.7%**	99.7%
Digits reversed	26%	26%	26%	26%
Letters between digits, hexadecimal, base64 of base64	0%	0%	0%	0%
Tricks fully caught	2 of 26	14	15	**22**
Real log lines changed (16,000)	95	95	95	95
Real SMS changed (2,412)	22	22	22	22

Two bugs found and fixed. The number patterns refused any number followed by a full stop, to skip decimals, so "mobile 9876543210." at the end of a sentence passed untouched (0 of 625 caught before the fix). And Python's `\d` matches Devanagari digits, so a number that mixed scripts reached the checksum code and crashed it on 125 of 500 lines when no normaliser was attached, which an attacker could use to knock the detector over. The patterns are now ASCII-only and rule out a full stop only when a digit follows it.

Three additions, off by default so E10 still measures the normaliser as built: number words in English, Hindi, Tamil and Hinglish, with "double" and "triple"; look-alike letters inside a mostly-digit run; and a second pass of escape decoding. They changed no real log line or SMS.

Left open on purpose: letters between digits, reversed digits, hexadecimal and nested base64. Undoing them on every line would flag ordinary numbers; an attacker who goes that far is better caught by the immune layer watching the flow (an unusual sink or volume) than by a pattern.

5. Speed (first step on items 7 and 12)

Profiling showed over 90% of the time inside the NER, mostly in building word features with regular expressions. Two changes in `ai/ner.py`: word features are cached per distinct word (same output), and `may_have_entities` is a cheap gate that sends a line to the NER only if it has Devanagari, a cue word, an address word, a weak cue such as "from" followed by a capital, or two capitalised words that are not dates.

E9 run, 16,000 lines	Lines per second	Values removed	Clean lines changed
NER on every line, features as before	274	91.0%	150

E9 run, 16,000 lines	Lines per second	Values removed	Clean lines changed
Features cached per word (output identical, checked)	348	91.0%	150
Cached features + NER gate	**654**	91.0%	**130**
No NER at all (upper bound)	864	52.7%	119

The gate sent 2,194 of 16,000 lines to the NER and skipped none of the 1,797 TEST lines that contain a name or address. A name in all lower case with no cue word would be skipped. The target of 10,000 lines per second per core still needs the Rust or vectorised port of item 12; the remaining time is the checksum, FF1 and regex stages and Python overhead per line.

What is next

1. Fix the two E13b failures (value before the OTP word; "no" as a mobile cue) and measure on a third set of unseen phrasings.
2. Run E14 against a real model through `openai_upstream`, to see whether the model rephrases placeholders.
3. Names are still the weakest type (77% in E9, and all 187 values that reached the provider in E14 were names or addresses): item 7, a transformer NER behind the new gate.
4. Item 12: port the checksum, FF1 and regex stages to vectorised code.
5. Items 8 (Tamil NER), 10 (OCR on KYC images), 11 (immune layer on templates) and 13 (paper and legal review) are not started.

Sources

- C. E. Shannon, "Communication Theory of Secrecy Systems", 1949; G. Smith, "On the Foundations of Quantitative Information Flow", FoSSaCS 2009.
- PCI Security Standards Council, PCI DSS v4.0, requirement 3.4.1 (display of the BIN and last four digits).
- Reserve Bank of India, FREE-AI framework, August 2025.
- OWASP Top 10 for LLM Applications 2025 (LLM01 prompt injection, LLM02 sensitive information disclosure).
- S. A. Crosby and D. S. Wallach, "Efficient Data Structures for Tamper-Evident Logging", USENIX Security 2009.
- Loghub: J. Zhu et al., ISSRE 2023. UCI SMS Spam Collection: T. Almeida et al., 2011.